

+1 Shrink Keyboard Escape Template

A complete, no-programming-required walkthrough for configuring, rebranding, and balancing your copy of the template — done entirely inside Roblox Studio.

This guide assumes no coding experience and no extra software. Every step tells you exactly which Explorer location to open, exactly what to change, and exactly what Studio screen to use. Anything that means editing a script uses a **SCRIPT** tag; anything done entirely inside Studio's Explorer/Properties uses a **STUDIO** tag; anything done on the Roblox website uses a **WEBSITE** tag.

Covers: opening your place file under your own group · enabling required Roblox services · setting your Group ID · replacing every monetization ID with your own · how the whole Shrink/Level/Wins/Rebirth/Worlds loop fits together · the Shrink (size) curve · Worlds 2 & 3 · Stage Gates · every balancing knob · Treadmills · Auto Wins · Global Leaderboards · admin access · the level-design tag reference · adding Trails and Auras. An optional appendix at the end covers Rojo, for anyone who prefers editing scripts in an external code editor.

Table of Contents

- 1** Getting the Project Open in Studio
- 2** Turning On the Services Your Game Needs
- 3** Setting Your Roblox Group
- 4** Replacing Every Monetization ID With Your Own
- 5** How the Game Works — The Core Loop
- 6** The Shrink System — Character Size & the Level Curve
- 7** Worlds — Balancing World 2 & World 3
- 8** Stage Gates — Level-Locked Barriers
- 9** Balancing the Game — Every Other Knob
- 10** Treadmills — Personal Speed Stations & Rebirth-Gated Tiers
- 11** Auto Wins — The Farm Toggle
- 12** Global Leaderboards
- 13** Giving Yourself Admin Access
- 14** Level-Design Building Blocks — Tag Reference
- 15** Adding a New Trail
- 16** Adding a New Aura
- 17** Pre-Publish Checklist
- A** Appendix — Optional: Editing With Rojo

1. Getting the Project Open in Studio

GOAL — Open your copy of the game, save it under your own Roblox group, and learn the one skill you'll reuse for every step in this guide: opening a script in the Explorer and editing it.

1.1 Open the place file STUDIO

You were provided a Roblox place file (`.rbxlx`). Double-click it — it opens directly in Roblox Studio, fully built and ready to edit. There is nothing to install and nothing to build.

1.2 Save it under your own group STUDIO

1. In Studio's **File** menu, choose **Save to Roblox As...**
2. In the dialog that appears, set the **creator** to your own Roblox **Group** (not your personal account) — this publishes the game as belonging to your group, which matters once you set up monetization and the group-join reward.
3. Give it a name and click **Save**. This is now your own, fully independent copy of the game.

✓ **TIP** — If you don't yet have a group, create one first at roblox.com — it takes a minute and is free.

1.3 How every step in this guide works STUDIO

This game's settings all live in small scripts called **ModuleScripts**, organized in folders inside Studio's **Explorer** window (open it via **View** → **Explorer** if you don't see it). Every step below gives you a Studio path like this:

Script to edit	ReplicatedStorage/Shared/Configs/GroupRewardsConfig
----------------	---

Read it left to right as folders to open in the Explorer tree: open `ReplicatedStorage`, then `Shared`, then `Configs`, then double-click `GroupRewardsConfig` to open it in Studio's built-in Script Editor. Make your edit right there, then save the whole place with **Ctrl+S** (or **File** → **Save to Roblox**) when you're done.

✓ **TIP** — That's the entire skill this guide relies on. Every chapter after this one is just "go to this path, change this value."

2. Turning On the Services Your Game Needs

GOAL — Enable the Roblox backend services this template relies on, so saving, leaderboards, and admin tools work both in Studio and after publishing.

This game saves player progress, runs global leaderboards, and syncs Admin Panel actions across servers. All three require one switch to be turned on.

2.1 Enable Studio Access to API Services STUDIO

1. In Studio, go to the **Home** tab → **Game Settings**.
2. Open the **Security** tab.
3. Turn on **Enable Studio Access to API Services**.
4. Click **Save**.

Without this, testing the game inside Studio will fail to save data or show leaderboards (you'll see red errors in the Output window mentioning `DataStoreService`).

2.2 What each service is used for (informational)

Player data saving (Shrink XP, Level, Wins, Rebirths, World, cosmetics, everything)	DataStoreService — used by the data-saving system. If this is off, nobody's progress saves.
Global Level, Wins & Time Played leaderboards (the signboards in the world)	DataStoreService — used by the Global Leaderboard system (Chapter 12).
Admin Panel actions (Shrink/Wins boosts, event treadmills, announcements) reaching every live server at once	MessagingService — this is on by default once API access is enabled; no separate switch.

⚠ IMPORTANT — Once you **publish** the game (not just test in Studio), these services work automatically — the Studio toggle above only affects testing inside the editor. You do not need to touch "Allow HTTP Requests" — this template makes no external web requests.

3. Setting Your Roblox Group

GOAL — Point the "join our group for a reward" feature at your own Roblox group instead of the template author's.

Players who join your group and click Claim in-game receive a one-time Shrink XP reward. This is controlled by a single number.

Script to edit

ReplicatedStorage/Shared/Configs/GroupRewardsConfig

```
GroupRewardsConfig.GroupId = 423275116 -- replace with your Group ID
GroupRewardsConfig.XpReward = 15000    -- how much Shrink XP the reward grants
```

1. **(Website)** Create (or open) your group on the Roblox website.
2. Your Group ID is the number in the group's URL, e.g. `roblox.com/groups/423275116/My-Group`.
3. Replace the number after `GroupRewardsConfig.GroupId =` with your group's number.
4. Optionally change `XpReward` to however much Shrink XP you want to grant (currently 15,000).

4. Replacing Every Monetization ID With Your Own

GOAL — Make every Robux purchase button in the game sell something YOU own, so the money goes to you.

⚠ IMPORTANT — Every ID currently in this script belongs to the template author's own game. If you publish without replacing them, purchases will either fail outright or attempt to sell items you do not own. This is the single most important step before launch.

Script to edit

ReplicatedStorage/Shared/Configs/MonetizationConfig

4.1 Important — everything here is a Developer Product, even the "Game Passes"

This template purchases *everything* — including permanent unlocks like Trails, Auras, Treadmill tiers, and Shrink Pass tiers — as Roblox **Developer Products**, not as real Roblox Game Passes. This is intentional: unlike Game Passes, Developer Products can be gifted to another player, which is what makes a future "gift this to a friend" feature possible. The game's own code (behind the scenes) treats a Developer Product listed under the `GamePasses` section as a *permanent* unlock the moment it's purchased once — from a player's point of view it behaves exactly like a normal Game Pass. You never need to create an actual Game Pass on the Creator Dashboard for anything in this game.

⚠ IMPORTANT — Wherever this guide (or the config script) says "GamePasses", create a **Developer Product** for it — not a Game Pass. This applies to every single entry in the file, in both the `GamePasses` section and the `DevProducts` section.

4.2 Step-by-step, for every single entry

1. **(Website)** Go to the **Creator Dashboard** for your own game.
2. Under **Monetization** → **Developer Products**, create one new Developer Product for every item listed in the tables below (name it however you like, set your own price), and copy its ID.
3. **(Script)** In `MonetizationConfig`, replace each number with the matching ID you just created. Keep the field *names* exactly as they are — only change the numbers.

4.3 Every permanent unlock (listed under `GamePasses` in the script)

Category	Entries (field names)	What it's for
<code>Treadmills</code>	<code>Gold</code> , <code>Diamond</code> , <code>Zombie</code> , <code>Admin</code>	Personal treadmill tiers a player can own (Chapter 10).
<code>Trails</code>	<code>Green</code> , <code>Blue</code> , <code>Purple</code> , <code>Red</code> , <code>Yellow</code>	Cosmetic trail unlocks (Chapter 15).
<code>Auras</code>	<code>Purple</code> , <code>Red</code> , <code>Yellow</code> , <code>Green</code> , <code>Blue</code>	Cosmetic aura unlocks (Chapter 16).
<code>ShrinkPass</code>	<code>Tier2</code> , <code>Tier4</code> , <code>Tier8</code> , <code>Tier16</code>	Permanent Shrink-XP-per-press multiplier tiers (2×/4×/8×/16×, each replacing the last).
<code>WinsMultiplier</code>	<code>Double</code>	Permanent 2× Wins unlock (every stage's normal Wins reward, everywhere).
<code>KeycapSounds</code>	<code>Unlock</code>	Unlocks all 8 premium keycap sounds at once.

4.4 Every repeatable purchase (listed under `DevProducts` in the script)

Category	Entries	What it's for
<code>RebirthSkip</code>	single ID	Skip the Level requirement for your next rebirth.
<code>SkipToStage</code>	<code>["2"]</code> through <code>["10"]</code>	Instantly teleport to that stage via Robux.
<code>ReviveToPreviousStage</code>	single ID	Revive-in-place after dying, back at the stage you died on.
<code>XpBoost</code>	3 unnamed entries (list positions 1-3, matching <code>XpBoostConfig</code> 's 10/30/60 minute tiers)	Temporary 2× Shrink boosts, increasing duration per tier.
<code>ServerBoost</code>	5 unnamed entries (2×, 4×, 6×, 8×, 10× — see code comments)	Server-wide Shrink multiplier, benefits every player on the server.
<code>PlusPercentShrink</code>	<code>FirstOption</code> (2%), <code>SecondOption</code> (5%), <code>ThirdOption</code> (12%)	One-time flat Shrink-XP bonus based on the buyer's current Shrink total.
<code>x2Win</code>	single ID, shared by every stage	Doubles the Wins earned from whichever win pad the player is currently standing at (Chapter 9.1).

✓ **TIP** — The `XpBoost` and `ServerBoost` entries are plain lists (no names) — replace the numbers in the same order, left to right, and don't change how many entries are in the list.

5. How the Game Works — The Core Loop

GOAL — A plain-English map of how every system in this game connects to every other one, so the balancing chapters that follow actually make sense. Nothing to configure here — just read it once.

5.1 The moment-to-moment loop

1. **Press keycaps.** Each stage's floor is tiled with keycaps (Chapter 14.5). Pressing one grants the player **Shrink** — this game's XP stat (internally still called `Shrink` in the data, shown to players as "Shrink").
2. **Shrink accumulates into Level.** `LevelConfig` defines how much Shrink each next Level costs (Chapter 9.3). Leveling up is announced with a popup showing the old/new "Shrink Score" (a cosmetic 1→1000 display number, unrelated to the raw Shrink stat — see Chapter 6).
3. **Level shrinks the character.** `ShrinkConfig` maps the player's current Level onto a character scale between 1.0 (full size) and 0.2 (fully shrunk) — see Chapter 6. This is the whole "+1 Shrink" hook: the higher you level, the smaller you get.
4. **Higher Level unlocks things.** Stage Gates open at Level thresholds (Chapter 8), Rebirth becomes available at a Level threshold (Chapter 9.2), and Worlds 2/3 unlock at Level thresholds (Chapter 7).
5. **Reaching a stage's win pad grants Wins** — the other currency, spent on cosmetics, teleport skips, and the press-multiplier stands next to the keycap floor (Chapter 9.1/9.6). Wins do not reset on Rebirth.
6. **Rebirth resets Level and Shrink back to 0** (character pops back to full size) **in exchange for a permanent Shrink-gain multiplier** that stacks with everything else (Chapter 9.2). This is the prestige loop.
7. **Worlds are separate zones, each with their own full stage/keycap/Wins/Shrink progression** layered on the same never-resetting Level stat (Chapter 7). Reaching World 2 or 3 doesn't reset Level — it just remaps that same Level onto a fresh 1.0→0.2 shrink curve and a fresh set of stage gates.

5.2 What multiplies what

Everything that boosts *Shrink gain per keycap press* stacks multiplicatively (see `ReplicatedStorage/Shared/Packages/XpPerPress`): Rebirth multiplier × Aura multiplier × Trail multiplier × personal Treadmill multiplier × temporary Shrink Boost (2× while active) × Shrink Pass tier × server-wide Server Boost × any Admin-set multiplier. Wins earned per stage is a separate number (`WinsConfig`) that only the `WinsMultiplier` gamepass, the per-touch `x2Win` purchase, and the Admin Wins multiplier affect.

✓ **TIP** — Every per-world config in this game (Wins, Stage Gates, Shrink curve, teleport prices, press-multiplier stands) is keyed by the player's current `World` value and falls back to World 1's table if a world has no entry — so it's always safe to leave World 2/3 half-tuned while you test, nothing will error.

6. The Shrink System — Character Size & the Level Curve

GOAL — Control exactly how small a player gets, how fast, and at what Level — independently for each of the 3 worlds.

Script to edit

ReplicatedStorage/Shared/Configs/ShrinkConfig

6.1 The big picture

A player's **Level** is one single, ever-increasing stat shared across every world (Rebirth is the only thing that resets it). Each world, however, maps that same Level onto its *own* shrink curve, over its own Level range:

```
ShrinkConfig.BaseScale = 1.0    -- full size, at or below a world's MinLevel
ShrinkConfig.MinScale  = 0.2    -- fully shrunk, at or above a world's MaxLevel
ShrinkConfig.MaxWorld  = 3

ShrinkConfig.LevelRangeByWorld = {
    [1] = { MinLevel = 1,    MaxLevel = 400 },
    [2] = { MinLevel = 1,    MaxLevel = 875 },
    [3] = { MinLevel = 1,    MaxLevel = 1800 },
}
```

Practically: a player who teleports to World 2 at Level 10 reads as full size again (Level 10 $\leq$ World 2's MinLevel-relative range start), then shrinks back down as their Level climbs toward World 2's own **MaxLevel** of 875. Raising a world's **MaxLevel** stretches out that world's shrink (and Shrink Score, see 6.3) progression over more Levels; lowering it compresses it into fewer.

6.2 The per-level scale tables

Below **LevelRangeByWorld**, **ScaleByLevelByWorld[1]** / **[2]** / **[3]** are large precomputed tables — one scale value per Level, linearly interpolated from 1.000 down to 0.200 across that world's range. This is intentional: it lets you hand-tune any single Level's exact scale (e.g. make Level 250 shrink a little extra for a "milestone" feel) by editing that one entry, without touching any formula.

```
-- World 1 excerpt:
ScaleByLevelByWorld[1] = {
    [1] = 1.000, [2] = 0.998, [3] = 0.996, ...
    [400] = 0.202, -- clamped to MinScale (0.2) by GetScaleForLevel once Level  $\geq$ 
    MaxLevel
}
```

⚠ **IMPORTANT** — These tables are generated, not hand-typed — if you change a world's `MinLevel` / `MaxLevel` range, the corresponding table needs to be regenerated to match (every Level in the new range needs an entry). If you're not comfortable writing a script for that, ask your developer to regenerate it, or keep the ranges as shipped and only hand-tune individual entries.

6.3 Shrink Score — the cosmetic level-up number

`ShrinkScoreByLevelByWorld[1]` / `[2]` / `[3]` is a second table, shown to the player on the level-up popup (Chapter 5.1) as a "Shrink Score" for that world, scaled off the player's position within that world's Level range. It has no gameplay effect; it exists purely so players have a clean, world-relative number to chase instead of a raw Level number that keeps climbing forever.

6.4 Shrink Pass — a purchasable multiplier, not a size boost

Despite the name, the `ShrinkPass` gamepass (Chapter 4.3) does not change character size directly — it multiplies Shrink-XP-per-press (Chapter 5.2), which makes the player Level up (and therefore shrink) faster. Configured in `ReplicatedStorage/Shared/Configs/ShrinkPassConfig`:

```
ShrinkPassConfig.Tiers = {  
    { Id = "Tier2",  Multiplier = 2 },  
    { Id = "Tier4",  Multiplier = 4 },  
    { Id = "Tier8",  Multiplier = 8 },  
    { Id = "Tier16", Multiplier = 16 },  
}
```

Owning a higher tier replaces the lower one (not additive) — order in this list matters, since the game walks it to find "the next tier this player doesn't own yet" for the shop UI.

7. Worlds — Balancing World 2 & World 3

GOAL — Understand how the 3-world structure is built in Explorer, and how to retune when each world unlocks.

7.1 Level requirement to unlock each world

Script to edit	ReplicatedStorage/Shared/Configs/TeleportWorldsConfig
<pre>TeleportWorldsConfig.LevelRequirement = { [1] = 0, [2] = 400, [3] = 875, }</pre>	

World 2 unlocks once a player has finished shrinking all the way through World 1 (Level 400, its **MaxLevel** — Chapter 6.1), and World 3 unlocks once they've finished World 2 (Level 875, World 2's own **MaxLevel**) — each world's requirement lines up with the previous world's shrink curve topping out, so a player always arrives at the next world already at Level 1 relative to that world's own range (full size again, per Chapter 6.1's per-world remap). Raise or lower these numbers if you'd rather gate world access at a different point in progress.

7.2 How a world is laid out in the Explorer **studio**

World 1 keeps every path this template originally shipped with, unprefixed, directly under **Workspace** (e.g. Workspace/Functional, Workspace/Decorations). World 2 and World 3 are each a self-contained copy of that same structure, nested one level deeper:

World 1	Workspace/Functional/...
World 2	Workspace/World2/Functional/...
World 3	Workspace/World3/Functional/...

Everything that's per-world at the Studio level — the spawn location, the Decorations/Stages folders used by Auto Wins (Chapter 11), the Stage Gates (Chapter 8) — follows this same "World1 stays at the root, World2/3 live under their own **WorldN** folder" convention. When building out World 2 or 3's physical level, duplicate World 1's **Functional** / **Decorations** structure into the matching **WorldN** folder rather than inventing a new layout.

7.3 The teleport pads between worlds **studio**

The actual "teleport to World N" destination points live at Workspace/Functional/WorldsTeleportPositions, as three Parts named **"1"**, **"2"**, and **"3"**. A player teleporting to a world is

moved to that world's matching named Part.

7.4 Everything else that's already per-world

Wins-per-stage (`WinsConfig.WinsByWorld`), teleport-skip prices (`TeleportConfig.StagesByWorld`), and press-multiplier stand thresholds (`XpPerPressConfig.OptionsByWorld`) are all separately keyed by world already — see Chapter 9 for tuning each. World 2/3's shipped numbers are placeholder progressions (roughly $\text{World } 1 \times 10 / \times 100$) — treat them as a starting point, not a balanced curve.

8. Stage Gates — Level-Locked Barriers

GOAL — Understand and retune the invisible walls that block a player from a stage until they've reached the right Level.

Script to edit

ReplicatedStorage/Shared/Configs/StageGates

8.1 How it works

Every gate Part in the world is tagged `StageGate` (via Studio's Tag Editor plugin — see Chapter 14) and named after the stage number it guards, e.g. `"5"`. On the client, its `CanCollide` is turned off the moment the player's Level meets that stage's requirement for their *current world* — this is purely a local physics check, so it updates instantly as the player levels up or changes world, with no server round-trip.

World 1 gates	Workspace/Functional/StageGates/World1/<stage number>
World 2 gates	Workspace/Functional/StageGates/World2/<stage number>
World 3 gates	Workspace/Functional/StageGates/World3/<stage number>

Which world's `LevelRequirementsByWorld` table a gate is checked against is driven entirely by the player's own current World value, not by the gate's location — but every gate should still be built under the matching `StageGates/WorldN` folder shown above, so the physical level layout for each world stays organized and matches the same per-world convention used everywhere else in Chapter 7.2 (Functional, Decorations, AutoWinsStart, etc.).

```
StageGates.LevelRequirementsByWorld = {  
  [1] = { ["1"] = 5, ["2"] = 10, ["3"] = 15, ... ["20"] = 400 },  
  [2] = { ["1"] = 1, ["2"] = 425, ... ["20"] = 875 },  
  [3] = { ["1"] = 1, ["2"] = 900, ... ["20"] = 1800 },  
}
```

Raise a stage's number to require more Level before that gate opens; lower it to open sooner. A stage name with no entry stays permanently closed (fail-safe) rather than defaulting to open, so double-check every gate you place in Studio has a matching entry here.

8.2 Placing a new gate STUDIO

1. Build the gate as a normal Part (or Model with a Part) spanning the path you want blocked.
2. Parent it under the matching world's folder, e.g. `Workspace/Functional/StageGates/World2` if it belongs to World 2.
3. Using Studio's Tag Editor plugin, add the `StageGate` tag to it.
4. Name the Part (or its tagged Part) to match the stage number key you want it to use, e.g. `"7"`.

5. Add a matching entry under that same world's table in `StageGates.LevelRequirementsByWorld`.

9. Balancing the Game — Every Other Knob

GOAL — A complete reference of every remaining number in the game that affects pacing, difficulty, or the in-game economy — where it lives, what it currently is, and what changing it does.

9.1 Wins earned per stage, per world

Script to edit

ReplicatedStorage/Shared/Configs/WinsConfig

```
WinsConfig.WinsByWorld = {
  [1] = { [1] = 1, [2] = 5, [3] = 25, [4] = 50, [5] = 100, ... [20] = 750_000 },
  [2] = { [1] = 1_000_000, ... [20] = 250_000_000 },
  [3] = { [1] = 300_000_000, ... [20] = 30_000_000_000 },
}
```

How many Wins a player earns for reaching each stage in each world. Raising a value speeds up how fast players can afford teleports and cosmetics; lowering it slows the whole economy down. A

RequireRobux attribute on a win pad's Part switches it to a Robux-only "2× Wins" pad instead (see the **x2Win** Developer Product, Chapter 4.4) — its label shows the Robux price automatically.

9.2 Rebirth tiers

Script to edit

ReplicatedStorage/Shared/Configs/RebirthConfig

```
RebirthConfig.Tiers = {
  [1] = { RequiredLevel = 10, Multiplier = 2 },
  [2] = { RequiredLevel = 20, Multiplier = 3 },
  [3] = { RequiredLevel = 40, Multiplier = 4 },
  ...
  [33] = { RequiredLevel = 1900, Multiplier = 110 },
  [34] = { RequiredLevel = 2000, Multiplier = 115, -- max tier
}
```

RequiredLevel is how high a player's Level must be to perform that rebirth (higher = harder to reach). **Multiplier** is the permanent Shrink-gain multiplier they keep afterward (stacks with everything in Chapter 5.2). Performing a rebirth resets both Level and Shrink to 0 (the character pops back to full size) and reloads the character. The highest-numbered tier is the maximum — players who exceed it stay at that tier's multiplier forever, and can still rebirth again for the same reward.

9.3 Level / Shrink-XP curve

Script to edit

ReplicatedStorage/Shared/Configs/LevelConfig

```
LevelConfig.BaseFactor = 12.968167082447044
LevelConfig.Exponent = 1.4
LevelConfig.MaxPrecomputedLevel = 2000
```

The Shrink XP needed to go from Level `n` to `n+1` is `round(BaseFactor * n ^ Exponent)`. Because `Exponent` is greater than 1, each step costs more than the last, and grows increasingly steeply the higher `n` gets — cheap early levels, a harder and harder climb later. Raising `BaseFactor` scales every level's cost up proportionally (the whole curve gets harder evenly); raising `Exponent` steepens the curve's shape instead (later levels get disproportionately harder relative to early ones). Levels up to `MaxPrecomputedLevel` are pre-calculated into a lookup table in this same script for performance — for hand-tuning specific individual levels, edit the table entries directly instead of the formula.

9.4 Stage teleport (skip) prices, per world

Script to edit

ReplicatedStorage/Shared/Configs/TeleportConfig

```
TeleportConfig.StagesByWorld = {
  [1] = {
    { TeleportTo = 2, WinsCost = 5, UIColor = Color3.fromRGB(60, 245, 255) },
    { TeleportTo = 3, WinsCost = 25, UIColor = Color3.fromRGB(60, 255, 140) },
    ... up to TeleportTo = 20 ...
  },
  [2] = { ... }, -- separate WinsCost progression for World 2
  [3] = { ... }, -- separate WinsCost progression for World 3
}
```

The Wins price to skip straight to that stage, for the player's current world (players can also pay Robux via the matching `SkipToStage` Developer Product — a single shared ID per stage number across all worlds, Chapter 4.4). Raising a stage's cost pushes players toward either grinding more or paying Robux; lowering it makes the grind path more attractive. `UIColor` only affects the generated menu frame's background tint.

9.5 Cosmetic prices & power (Trails / Auras)

Trail	Wins Cost	Shrink Multiplier	Walk Speed
Green	500	1.5×	30
Blue	50,000	2×	35
Purple	5,000,000	3×	40
Red	50,000,000	4×	45
Yellow	5,000,000,000	5×	50

Aura	Wins Cost	Shrink Multiplier
Purple	15,000	2×
Red	15,000,000	4×
Yellow	150,000,000	6×
Green	5,000,000,000	8×
Blue	50,000,000,000	10×

Trail and Aura multipliers stack multiplicatively with everything else (Chapter 5.2) while equipped. Trails additionally set the player's `Humanoid.WalkSpeed` outright (not a bonus on top — the equipped trail's `WalkSpeed` replaces it, or `TrailsConfig.BaseWalkSpeed = 25` if nothing is equipped) — this is the only system in the game that touches `WalkSpeed`. Raising `WinsCost` in either script makes that cosmetic more expensive; raising `Multiplier` makes it more powerful while equipped. Scripts: `ReplicatedStorage/Shared/Configs/TrailsConfig` and `ReplicatedStorage/Shared/Configs/AurasConfig`.

9.6 Press-multiplier stands (Shrink-per-keycap-press options)

Script to edit	<code>ReplicatedStorage/Shared/Configs/XpPerPressConfig</code>
----------------	--

Matches Parts named `"1"` through `"8"` (or more) under `Workspace/Functional/XpPerPressOptions`, one set per world, each with a `World` attribute. Standing on/touching one equips it if the player has enough Wins:

```
XpPerPressConfig.OptionsByWorld[1] = {
    [1] = { StepAmount = 1,    WinsRequired = 0 },
    [2] = { StepAmount = 2,    WinsRequired = 3 },
    ...
    [18] = { StepAmount = 1000, WinsRequired = 3_500_000 },
}
```

`StepAmount` is the raw Shrink XP granted per keycap press while that option is equipped (before any of the Chapter 5.2 multipliers are applied); `WinsRequired` is the Wins total needed to unlock it. The game automatically keeps a player equipped on the *highest* option they can currently afford, and re-checks this any time their Wins changes (earned or spent) — you never need to touch the equipped-option logic itself, just these two numbers per tier.

9.7 Flat Shrink-bonus purchase tiers

Script to edit	<code>ReplicatedStorage/Shared/Configs/PlusPercentShrinkConfig</code>
----------------	---

```
FirstOption  = 0.02  -- 2%
SecondOption = 0.05  -- 5%
ThirdOption  = 0.12  -- 12%
```

Each purchase grants a one-time flat Shrink XP bonus equal to this percentage of the player's *current* Shrink stat at the moment of purchase (Chapter 4.4's `PlusPercentShrink` Developer Products). Raising these numbers makes each purchase grant a bigger one-time jump.

9.8 Temporary Shrink boosts

Script to edit

ReplicatedStorage/Shared/Configs/XpBoostConfig

```
{ DurationMinutes = 10 }, { DurationMinutes = 30 }, { DurationMinutes = 60 }
```

Each entry auto-generates one tier in the Shop's "2× Shrink" section, always a flat 2× multiplier for that long — only the duration differs per tier. Each entry needs a matching

`MonetizationConfig.DevProducts.XpBoost` price (Chapter 4.4, matched by list position, not by duration).

9.9 Keycap sounds (cosmetic only)

ReplicatedStorage/Shared/Configs/KeycapSoundConfig — purely cosmetic, no balance impact. Each sound has a `Free` flag; non-free ones unlock together via the single `KeycapSounds.Unlock` entry from Chapter 4.3.

10. Treadmills — Personal Speed Stations & Rebirth-Gated Tiers

GOAL — Understand the personal treadmill system players can spawn to passively earn Shrink XP, including tiers gated by Rebirth count instead of (or in addition to) a gamepass.

Script to edit

ReplicatedStorage/Shared/Configs/TreadmillConfig

10.1 How it works

A player can spawn their own personal treadmill near themselves (bound to a UI toggle, firing the `ToggleSpawn` remote); standing on any treadmill (their own, or one placed permanently in the world under `Workspace/Functional/Treadmills`) grants Shrink XP every `XpTickInterval` seconds (default 0.05s) at that treadmill's `Multiplier`. Only one personal treadmill exists per player at a time — spawning a new one, or walking off, despawns the old one automatically.

10.2 Treadmill tiers & gating STUDIO

Personal treadmill tiers are Models under `ReplicatedStorage/Shared/Assets/PlayerTreadmills`, each carrying Attributes (Properties panel → Attributes → +):

Attribute	Type	Meaning
<code>Multiplier</code>	Number	Shrink-gain multiplier while standing on this tier.
<code>RequiresGamePass</code>	Boolean	If true, the player must own the matching gamepass to use this tier.
<code>GamePassName</code>	String	Must match a key under <code>MonetizationConfig.GamePasses.Treadmills</code> (Chapter 4.3), e.g. <code>"Gold"</code> .
<code>RebirthRequirement</code>	Number (optional)	Minimum Rebirth count required to use this tier — this is the "Rebirth Treadmill" gating. Can be combined with <code>RequiresGamePass</code> for a tier that needs both.

The game always spawns the *highest-Multiplier* tier a player is currently eligible for. A player who doesn't yet own a required gamepass gets prompted to buy it (rate-limited to once every few seconds); a player short on Rebirths gets a "Need N Rebirth" notification instead.

10.3 Admin-spawned event treadmills

Temporary, ungated, world-wide treadmills any player can stand on — spawned via the Admin Panel's `SpawnEventTreadmill` action (Chapter 13) for a limited duration, always replacing any currently-active event treadmill. Templates live under `ReplicatedStorage/Shared/Assets/EventTreadmills` (each just needs a `Multiplier` attribute, no gamepass gating), and spawn at a single fixed Part: `Workspace/Functional/EventTreadmillSpawnCFrame`. Two ship with the template out of the box — `UFO` and `Radioactive` — and the Admin Panel's action takes that exact name (case-insensitive) to pick which

one to spawn; add more by dropping a new Model with a **Multiplier** attribute into the same folder.

11. Auto Wins — The Farm Toggle

GOAL — Understand the "Auto Wins" button players can toggle to have their character automatically walk back and forth to the best win pad they've unlocked, and what Studio structure it depends on.

11.1 What it does

Toggling the button (Main/RightIcons/AutoWins/Button in the player's UI) fires the `SetAutoWins` remote. While enabled, the server walks the player's character between their current world's `AutoWinsStart` position and the win pad of the *highest stage gate they've already unlocked* (Chapter 8), banking Wins on every lap. Any manual movement input from the player immediately force-disables it client-side (they've taken control back).

11.2 Required Studio structure STUDIO

Farm start position	Workspace/Functional/AutoWinsStart/1 (or /2, /3)
Per-stage win pads	.../Stages/StageN/Wins/FreeWins and .../Stages/StageN/Wins/2xWins

`AutoWinsStart` is a Folder containing one BasePart per world, named `"1"`, `"2"`, `"3"` — this is where the character starts each farming lap and where it's teleported after every win pad touch (Chapter 9.1's `grantStageWins`). Each Stage folder (found via the same per-world Stages root as Chapter 7.2) needs a `Wins` subfolder containing a `FreeWins` Part and a `2xWins` Part — Auto Wins walks to the midpoint between the two first, then on to `FreeWins` specifically (it never auto-triggers the Robux-only 2× pad).

12. Global Leaderboards

GOAL — Understand the three cross-server signboards and how to retune their refresh rates.

Script to edit

ReplicatedStorage/Shared/Configs/LeaderboardConfig

```
{
    TopCount = 50,           -- entries shown per board
    StatSyncInterval = 30,   -- seconds between pushing a player's changed stat
    DisplayRefreshInterval = 60, -- seconds between re-fetching top N and redrawing
}
```

12.1 Required Studio structure STUDIO

Level board	Workspace/Functional/GlobalLeaderboards/Level/ LevelLeaderboard
Wins board	Workspace/Functional/GlobalLeaderboards/Wins/WinsLeaderboard
Time Played board	Workspace/Functional/GlobalLeaderboards/TimePlayed/GuiPart

Each board Part needs a `SurfaceGui` containing a `ScrollingFrame` with a `UIGridLayout` whose `Template` row has `TextLabel` (rank), `PlayerName`, and `AmountofCash` children — copy an existing board's structure exactly when placing a new one. Ranking uses a real cross-server `OrderedDataStore` per board (via `DataStoreService`, Chapter 2), so results are accurate across every live server, not just the one a player happens to be on. Lower the two interval numbers for a more "live-feeling" board at the cost of more DataStore calls (Roblox rate-limits these — don't drop far below the shipped defaults with a large player base).

13. Giving Yourself Admin Access

GOAL — Unlock the in-game Admin Panel (press P) for yourself, so you can broadcast announcements and grant temporary server-wide boosts while testing or running events.

Script to edit

ReplicatedStorage/Shared/Configs/AdminConfig

```
local AdminConfig = {}

AdminConfig.WHITELIST_ACTIVE = false -- true = only whitelisted players see the
Admin Panel
AdminConfig.Whitelist = {
    4176117504 -- Put your id here
}

function AdminConfig.IsWhitelisted(player)
    return table.find(AdminConfig.Whitelist, player.UserId) ~= nil
end

return AdminConfig
```

1. **(Website)** Find your numeric Roblox User ID: open your own Roblox profile page — the number in the URL (`roblox.com/users/4176117504/profile`) is your User ID.
2. **(Script)** Add a new line inside the `Whitelist = { ... }` list with your User ID, e.g.:

```
AdminConfig.Whitelist = {
    4176117504,
    YOUR_USER_ID_HERE,
}
```

3. Set `AdminConfig.WHITELIST_ACTIVE = true` once you're ready to restrict the panel to only whitelisted players. Leave it `false` during your own testing if you want anyone to be able to open it.

⚠ IMPORTANT — Always set `WHITELIST_ACTIVE = true` before your public launch — otherwise every player can open the Admin Panel and grant themselves boosts.

In-game: press **P** to open the Admin Panel once whitelisted. It currently exposes four actions, each broadcast instantly to every live server (not just yours):

Action	What it does
Set Shrink Multiplier	Server-wide temporary Shrink-gain multiplier, for a set duration (seconds).
Set Wins Multiplier	Server-wide temporary Wins multiplier, for a set duration (seconds).
Spawn Event Treadmill	Spawns a named treadmill from <code>ReplicatedStorage/Shared/Assets/EventTreadmills</code> (Chapter 10.3) for a set duration.
Announce	Broadcasts a chat/UI announcement to every player on every server, attributed to your name.

14. Level-Design Building Blocks — Tag Reference

GOAL — A cheat sheet of every CollectionService tag this game reacts to, so you can build a brand-new stage entirely inside Studio without touching a single script.

Apply tags using Studio's built-in **Tag Editor** plugin (Toolbox → search "Tag Editor", or **Model** → **Tag Editor** depending on your Studio version): select a Part, open the plugin, type the tag name, click Add.

Tag	Where it goes	What it does
WinHitbox	A win-pad Part, with Stage and World Number attributes set	Touching it grants that stage's Wins (Chapter 9.1). Add a RequireRobux Boolean attribute to make it a paid 2× pad instead.
StageGate	A barrier Part, named to match a stage number key	Blocks movement until the player's Level clears that stage's requirement for their current world (Chapter 8).
BaseXpOptionPart	A press-multiplier stand Part, with a World attribute, named "1" - "8"	Touching it equips that Shrink-per-press tier if the player can afford it (Chapter 9.6).
GenerateKeycapsSurface	A flat Part covering the area you want tiled with keycaps	Auto-generates a grid of pressable keycaps across the surface at game start — see 14.5 below.

Note: **Treadmill** and **EventTreadmill** tags (Chapter 10) are applied automatically by the game at startup to everything already parented under Workspace/Functional/Treadmills — you never need to hand-tag those.

14.5 Building a new keycap surface (the core "keyboard" gameplay) STUDIO

1. Build a flat Part sized to cover the floor area you want tiled with keycaps.
2. Tag it **GenerateKeycapsSurface**.
3. Add a **KeycapSet** String attribute naming a folder under ReplicatedStorage/Shared/Assets/Keycaps (each such folder is a themed set of keycap Models to randomly pick from while filling the grid).
4. Optionally add a **KeycapSizeMulti** Number attribute (defaults to 1) to scale every generated keycap up or down — useful for matching a stage's visual scale.

At game start, the surface is automatically divided into a grid sized to fit as many keycaps as will comfortably tile it, each cloned from a random Model in the chosen set, stamped with a random letter decal, and tagged **Keycap** so the press-detection system picks it up — nothing else needs to be built or wired by hand.

15. Adding a New Trail

GOAL — Add a brand-new equippable trail cosmetic that players can buy with Wins (in-game currency) and, optionally, Robux.

Trails are identified everywhere by a single short **id** (e.g. `Green`, `Blue`) which must be spelled *identically* in three places. Pick a new id first, e.g. `Orange`, and use it exactly everywhere below.

Step 1 — Duplicate the Studio asset STUDIO

1. In the Explorer, go to `ReplicatedStorage/Shared/Assets/Trails`.
2. Right-click an existing trail folder (e.g. `Green`) → **Duplicate**.
3. Rename the duplicate to your new id, e.g. `Orange`.
4. Inside it you'll find two `Attachment` instances named `Attachment0` and `Attachment1`, and a `Trail` instance named `TrailEffect`. **Keep these three names exactly as they are** — only change cosmetic properties like the Trail's `Color` or `Texture`. Their positions are recalculated automatically to fit whatever size the wearer currently is (Chapter 6), so you don't need to reposition them for different character scales.

Step 2 — Add the config entry SCRIPT

Script to edit

`ReplicatedStorage/Shared/Configs/TrailsConfig`

```
TrailsConfig.Items = {
  Green = { Name = "Green Trail", WinsCost = 500, Multiplier = 1.5, WalkSpeed =
30, Color = Color3.fromRGB(60, 220, 90) },
  -- ... existing entries ...

  Orange = { Name = "Orange Trail", WinsCost = 100_000_000, Multiplier = 6,
WalkSpeed = 55, Color = Color3.fromRGB(255, 140, 0) },
}
```

The key (`Orange`) must exactly match the Studio folder name from Step 1. `WinsCost` is the in-game currency price; `Multiplier` is how much it boosts Shrink XP per press while equipped (Chapter 5.2); `WalkSpeed` is the flat `Humanoid.WalkSpeed` the player gets while it's equipped (Chapter 9.5); `Color` is only used for the UI swatch.

Step 3 — (Optional) allow buying it with Robux SCRIPT

Create a Developer Product for it (see Chapter 4 — not a Game Pass), then add it under

`GamePasses.Trails` in `ReplicatedStorage/Shared/Configs/MonetizationConfig` using the same id:

```
Trails = {  
  Green = 3609493687,  
  -- ...  
  Orange = YOUR_NEW_DEVPRODUCT_ID,  
},
```

✓ **TIP** — Skipping Step 3 is fine — the trail will still be fully purchasable with Wins. The Robux buy button just won't work until you add this.

16. Adding a New Aura

GOAL — Add a brand-new equippable full-body particle aura, following the same id-matching pattern as Trails.

Step 1 — Duplicate the Studio asset STUDIO

1. Go to ReplicatedStorage/Shared/Assets/Auras.
2. Duplicate an existing aura (e.g. `Blue`) and rename it to your new id, e.g. `Orange`.

⚠ IMPORTANT — Each Aura template must be a **Part** containing a **ParticleEmitter** — not a bare ParticleEmitter on its own. When equipped, the game automatically resizes and repositions this Part to cover the wearer's whole body based on their *current* character size (Chapter 6), and the ParticleEmitter fills that volume. Duplicating an existing Aura keeps this working correctly automatically — just make sure you duplicate the whole Part (not just the ParticleEmitter inside it).

On the ParticleEmitter inside your duplicated Part, freely change **Color**, **Texture**, and **Rate** to design your look — but leave **Shape** (Box), **ShapeStyle** (Volume), and **LockedToPart** (On) exactly as they are, or the full-body coverage will break.

Step 2 — Add the config entry SCRIPT

Script to edit	ReplicatedStorage/Shared/Configs/AurasConfig
----------------	--

```
AurasConfig.Items = {
    Purple = { Name = "Purple Aura", WinsCost = 15_000, Multiplier = 2, Color =
Color3.fromRGB(170, 80, 255) },
    -- ... existing entries ...

    Orange = { Name = "Orange Aura", WinsCost = 500_000_000_000, Multiplier = 12,
Color = Color3.fromRGB(255, 140, 0) },
}
```

Step 3 — (Optional) allow buying it with Robux SCRIPT

Same as Trails: create a Developer Product (Chapter 4), then add it under `GamePasses.Auras` in ReplicatedStorage/Shared/Configs/MonetizationConfig with the matching id.

17. Pre-Publish Checklist

Work through this list before you make the game public.

- ☐ Place saved under your own Roblox group (Chapter 1), and opens with no red errors in the Output window.
- ☐ **Game Settings** → **Security** → **Enable Studio Access to API Services** is turned on.
- ☐ `GroupRewardsConfig.GroupId` points at your own group (Chapter 3).
- ☐ Every ID in `MonetizationConfig` has been replaced with a Developer Product you created in your own game (Chapter 4) — none are left as the template author's original numbers, and none were created as Game Passes.
- ☐ Balance values (Chapters 6, 7, 8, 9) reviewed and adjusted to your liking — the shipped defaults, especially World 2/3's placeholder economy numbers, are a starting point, not a finished balance pass.
- ☐ All required Studio Explorer structure exists: per-world `SpawnLocation`s (Chapter 9.1/Chapter 7.2), `AutoWinsStart` and per-world Stage `Wins` folders (Chapter 11.2), the three `GlobalLeaderboards` boards (Chapter 12.1), and every `StageGate`-tagged Part has a matching config entry (Chapter 8).
- ☐ Your own User ID is in `AdminConfig.Whitelist`, and `WHITELIST_ACTIVE` is set to `true` (Chapter 13).
- ☐ Any new Trails/Auras/stages you've added show up correctly and are equip/purchase-tested with a second test account (cosmetics equipped by one account shouldn't appear on another's screen incorrectly).
- ☐ Play-tested a full loop end-to-end: press keycaps → level up and watch the character shrink → earn Wins → buy a cosmetic → rebirth → unlock and teleport to World 2 → teleport to a later stage → toggle Auto Wins → open the Admin Panel (if whitelisted) → confirm the global leaderboards populate after a minute or two.

✓ **TIP** — Keep this guide — every chapter above maps to exactly one Explorer location, so you can return to it any time you want to add more content or retune the economy.

Appendix — Optional: Editing With Rojo

GOAL — Everything in this guide works entirely inside Studio — you never need this appendix. It's here only for developers who prefer editing scripts in an external code editor (like VS Code) with live sync back into Studio, e.g. for using version control or a team workflow.

This project's source is organized in a plain folder structure that a tool called **Rojo** can sync into Studio automatically. If you'd like to use it:

1. Install the Rojo command-line tool (see rojo.space) — this project includes an `aftman.toml` file, so if you use Aftman, running `aftman install` in the project folder installs the exact version needed automatically.
2. Install the **Rojo** plugin for Roblox Studio (search "Rojo" in Studio's Toolbox).
3. From a terminal in the project folder, run `rojo serve`.
4. In Studio, open the **Plugins** tab, click **Rojo**, then click **Connect**.
5. From then on, saving a script file in your code editor updates Studio automatically within a second or two.

✓ **TIP** — This is purely a convenience for people who already work in an external editor — it changes nothing about how the game itself behaves, and every script name/path referenced throughout this guide is identical whether you edit it via Rojo or directly in Studio.